

GRAFT – A RAFT LIBRARY

DEVELOPER MANUAL

June 13, 2026

Contents

1	Introduction	7
1.1	What this manual covers	7
1.2	Terminology.....	7
2	Design intent	8
3	Getting Started	9
3.1	Building from source	9
3.2	Starting a local cluster	9
3.3	First commands	11
3.4	Docker deployment.....	11
3.5	DNS SRV bootstrapping	11
3.6	Where to go next.....	13
4	System Overview	15
4.1	Java module map.....	15
4.2	C++ layout	15
4.3	Rust layout	16
4.4	Boundary principle	16
4.5	Shared protocol	16
5	Application Developer Guide	17
5.1	What an application must provide	17
5.2	Command design.....	17
5.3	State-machine contract.....	17
5.4	Snapshots	18
5.5	Reads and read leases.....	18
5.6	Client-facing policies	18
5.7	Cluster configuration storage	19

6	Consistency Boundaries	20
6.1	What “split brain” means here	20
6.2	Why “majority” and “minority” are well-defined	20
6.3	What Raft guarantees	21
6.4	What Raft does not guarantee	21
6.5	The three outcome classes	22
6.6	Where inconsistency can still appear	22
6.7	What application developers must accommodate	23
6.8	Recommended documentation for applications	23
6.9	Bottom line	24
7	Raft and XATMI Transaction Thinking	25
7.1	Short version	25
7.2	Where they are similar	25
7.3	Where they differ	25
7.4	Commit semantics	26
7.5	Ambiguous outcomes	26
7.6	Idempotency and request identity	26
7.7	Determinism	26
7.8	External resources	27
7.9	Mapping concepts	27
7.10	Rollback and compensation	28
7.11	Reads and lookup workloads	28
7.12	Cluster configuration is not domain data	28
7.13	Transaction boundaries for domain developers	28
7.14	Design guidance	29
7.15	Practical rule	29
8	Java Library	30
8.1	Application integration path	30

8.2	State machine interfaces.....	30
8.3	Runtime modules	30
8.4	Policy hooks.....	31
8.5	Storage	31
8.6	Javadocs and Maven publishing	31
9	C++ Library.....	32
9.1	Build model	32
9.2	Application boundary.....	32
9.3	Runtime and transport	32
9.4	Telemetry and observability	33
9.5	Protobuf toolchain.....	33
9.6	Parity expectations	34
10	Rust Library	35
10.1	Build model	35
10.2	Application boundary.....	35
10.3	Runtime and transport	36
10.4	Cross-language mixed smoke tests.....	36
10.5	Protobuf toolchain	36
10.6	Telemetry and observability	36
10.7	Policy hooks	37
10.8	Docker support	37
10.9	Parity expectations	38
11	Raft Internals	39
11.1	Core state	39
11.2	Elections.....	39
11.3	AppendEntries and commit.....	39
11.4	Read lease and read barrier	40
11.5	Joint consensus	40
11.6	Internal commands	40

12	Wire, Storage and Snapshots	41
12.1	Wire protocol.....	41
12.2	Status vocabulary	41
12.3	Persistent state.....	41
12.4	Snapshot wrapping	41
12.5	Configuration storage.....	42
12.6	Protocol evolution.....	42
13	Testing and Operations	43
13.1	Test layers	43
13.2	Java test workflow.....	43
13.3	C++ test workflow	43
13.4	Rust test workflow	43
13.5	Mixed Java/C++/Rust checks	44
13.6	Jepsen	44
13.7	Operational notes	44
13.8	Cross-language parity.....	45
13.9	Codebase navigation.....	45
14	Jepsen Testing	47
14.1	Architecture	47
14.2	Node implementations	47
14.3	Workload model	47
14.4	Nemesis catalog	48
14.5	Adding a nemesis	48
14.6	Reading failure output	48
14.7	Suite structure	49
14.8	Docker/SRV Jepsen	49
15	Appendix: Commands and Checklists	50
15.1	Repository build commands	50

15.2 Application developer checklist 50

15.3 Raft machinery developer checklist..... 50

15.4 Common failure interpretations 51

1 Introduction

This manual describes the Raft library from two developer viewpoints.

The first viewpoint is the application developer who wants to add cluster capability to a domain application. In that role the developer should not need to understand every election rule or replication edge case. The important questions are where application state enters the replicated log, how reads are made safe, how commands and snapshots are encoded, and what the Java, C++ and Rust integration surfaces expect.

The second viewpoint is the Raft machinery developer who maintains the library itself. In that role the developer must reason about terms, voting, leader state, log matching, snapshots, read leases, joint consensus, wire compatibility and cross-language parity. The Java, C++ and Rust implementations are intended to be functionally equivalent at the Raft boundary even though the language idioms, runtime frameworks and internal structure differ.

The library exists to let applications run in a Raft cluster without embedding domain knowledge in the consensus machinery. Reference-data distribution is an important example: writes are relatively rare and governed, while lookups are high-volume and need to happen close to consumers. The same library shape can also support other master-data or configuration-data applications where a central management path must feed fast replicated lookup state.

1.1 What this manual covers

The manual is organized around the boundary between domain application code and Raft internals:

- Section 3 walks through building and running a cluster.
- Section 4 summarizes the architecture and module layout.
- Section 5 explains how an application uses the library.
- Section 6 covers the split-brain, partition, and consistency guarantees an application can rely on.
- Section 7 maps Raft concepts to XATMI/XA transaction thinking for developers coming from that background.
- Sections 8, 9 and 10 describe the Java, C++ and Rust integration points.
- Section 11 explains the consensus machinery for maintainers.
- Section 12 covers protocol, persistence and snapshots.
- Section 13 explains test strategy and operational checks.
- Section 14 documents the Jepsen fault-injection framework.

1.2 Terminology

This manual uses *application* for the domain-specific code that owns business state and command encoding. It uses *Raft machinery* for elections, log replication, membership, snapshot metadata, read-safety decisions and transport handling. The boundary is deliberate: applications define what a command means; Raft decides when that command is committed and safe to apply.

2 Design intent

The project should be read as a library project, not as a single reference-data product. Reference data is a useful and important application, but the reusable asset is the cluster capability:

- replicate a deterministic state machine,
- preserve safety across leadership changes,
- expose typed client command and query requests,
- handle cluster membership through replicated configuration entries,
- support Java, C++ and Rust applications through equivalent protocol behavior.

The Java implementation is the most comprehensive implementation. The C++ and Rust implementations are developed as peer implementations and should be brought up to the same behavioral maturity where differences affect Raft semantics. Domain examples may differ temporarily, but the Raft machinery should not.

3 Getting Started

This section walks through bringing up a three-node Raft cluster and issuing the first commands. It is intended as an orientation for both application developers evaluating the library and machinery developers who need a working cluster to test against.

All three implementations (Java, C++, Rust) are covered. Pick the one that matches your build toolchain, or mix them using the Docker Compose files described in §3.4.

3.1 Building from source

3.1.1 Java

```
1 # Build all Java modules, run unit tests, produce raft-dist/target/raft-*.jar
2 mvn -q package
```

Listing 1: Java build

The distributable JAR lives at `raft-dist/target/raft-1.0-SNAPSHOT.jar`. It bundles all runtime dependencies and can be invoked directly:

```
1 java -jar raft-dist/target/raft-1.0-SNAPSHOT.jar nl@127.0.0.1:11081
```

Listing 2: Java CLI

3.1.2 C++

```
1 cmake -S graft-cpp -B graft-cpp/build
2 cmake --build graft-cpp/build
```

Listing 3: C++ build

Produces `raft-cpp/build/raft_smoke`, a single binary covering all server, client and probe commands.

3.1.3 Rust

```
1 cd graft-rust
2 cargo build -p graft-app-kv
```

Listing 4: Rust build

Produces `raft-rust/target/debug/raft-kv` (or `target/release/raft-kv` with `-release`). The binary provides 26 subcommands covering server, client, operational and smoke modes.

3.2 Starting a local cluster

The simplest way to start a cluster is with the active persistent server mode. Each node needs a unique port and a peer list containing all other nodes.

3.2.1 Java cluster

Start three terminals (or background processes):

```
1 java -jar raft-dist/target/raft-1.0-SNAPSHOT.jar \  
2   n1@127.0.0.1:11081 n2@127.0.0.1:11082 n3@127.0.0.1:11083  
3  
4 java -jar raft-dist/target/raft-1.0-SNAPSHOT.jar \  
5   n2@127.0.0.1:11082 n1@127.0.0.1:11081 n3@127.0.0.1:11083  
6  
7 java -jar raft-dist/target/raft-1.0-SNAPSHOT.jar \  
8   n3@127.0.0.1:11083 n1@127.0.0.1:11081 n2@127.0.0.1:11082
```

Listing 5: Java 3-node cluster

3.2.2 Rust cluster

The Rust CLI uses explicit `serve-active-persistent` with positional arguments instead of a combined `peer-spec/as`-argument convention:

```
1 graft-rust/target/debug/graft-kv serve-active-persistent \  
2   127.0.0.1 11081 n1 /tmp/n1.state 0 0 0 \  
3   n2@127.0.0.1:11082 n3@127.0.0.1:11083  
4  
5 graft-rust/target/debug/graft-kv serve-active-persistent \  
6   127.0.0.1 11082 n2 /tmp/n2.state 0 0 0 \  
7   n1@127.0.0.1:11081 n3@127.0.0.1:11083  
8  
9 graft-rust/target/debug/graft-kv serve-active-persistent \  
10  127.0.0.1 11083 n3 /tmp/n3.state 0 0 0 \  
11  n1@127.0.0.1:11081 n2@127.0.0.1:11082
```

Listing 6: Rust 3-node cluster

3.2.3 Verifying cluster health

Once all nodes are started, one will be elected leader within a few seconds. Verify with the `telemetry` or `cluster-summary` commands (both work against any implementation since they share the same wire protocol):

```
1 graft-cpp/build/graft_smoke cluster-summary 127.0.0.1 11081 cpp-cli  
2 graft-cpp/build/graft_smoke telemetry 127.0.0.1 11082 true
```

Listing 7: Cluster health check (C++ probe)

Or using the Rust binary:

```
1 graft-rust/target/debug/graft-kv cluster-summary 127.0.0.1 11081  
2 graft-rust/target/debug/graft-kv telemetry 127.0.0.1 11082 \  
3   --require-leader-summary
```

Listing 8: Cluster health check (Rust probe)

Look for state: LEADER on one node and state: FOLLOWER on the others.

3.3 First commands

Put a value and read it back. The client must target the leader; followers redirect.

```
1 graft-cpp/build/graft_smoke client-put 127.0.0.1 11081 greeting hello my-client
2 graft-cpp/build/graft_smoke client-get 127.0.0.1 11081 greeting my-client
```

Listing 9: Put and get (C++ probe)

Or with Rust:

```
1 graft-rust/target/debug/graft-kv client-put 127.0.0.1 11081 \
2   greeting hello my-client
3 graft-rust/target/debug/graft-kv client-get 127.0.0.1 11081 \
4   greeting my-client
```

Listing 10: Put and get (Rust probe)

If you send a client-get to a follower, the response will include `status: REDIRECT` with the leader's address.

3.4 Docker deployment

For multi-language clusters or reproducible testing, use Docker Compose. Four Compose files are provided:

- `docker/compose-srv-java.yml` — 3 Java nodes and 1 CoreDNS
- `docker/compose-srv-cpp.yml` — 3 C++ nodes and 1 CoreDNS
- `docker/compose-srv-rust.yml` — 3 Rust nodes and 1 CoreDNS
- `docker/compose-srv-mixed.yml` — 1 Java, 1 C++, 1 Rust and 1 CoreDNS

Start a cluster with:

```
1 docker/run-srv-smoke.sh java           # or cpp, rust, mixed
2 docker/run-srv-suite.sh               # all four in sequence
```

Listing 11: Docker smoke test

The smoke script builds images, starts containers, waits for leader election, probes all three nodes, and tears down on exit. All nodes share a CoreDNS container resolving the `raft.local` zone. Host ports 17001–17003 map to container port 7000.

3.5 DNS SRV bootstrapping

The local cluster examples above use explicit peer lists (`n2@127.0.0.1:11082`, `n3@...`) so each node knows its peers at startup. In containerized deployments—Docker Compose, Kubernetes, OpenShift—this becomes unwieldy. DNS SRV records let nodes discover peers from a single service name instead of a static list.

3.5.1 Why DNS SRV?

A DNS SRV record maps a service name to multiple *target* entries, each specifying a hostname and port. This matches the problem of cluster bootstrapping: a new node needs to contact *any* existing member, and the DNS zone is the natural place to publish the current member set.

SRV records are used natively in Kubernetes and OpenShift via headless Services. A StatefulSet-backed Raft cluster can publish SRV records automatically, giving each node a stable identity (`raft-0.raft.<ns>.svc.cluster.local`) and a discoverable service endpoint (`_raft._tcp.raft.<ns>.svc.cluster.local`). The same mechanism works in Docker Compose with a CoreDNS sidecar.

The startup contract is intentionally minimal. A node needs only three pieces of information:

- `RAFT_NODE_ID` — its logical Raft identity,
- `RAFT_BIND_HOST/RAFT_BIND_PORT` — its listen address,
- `RAFT_CLUSTER_SRV` — the SRV service name to resolve.

Everything else—current membership, leader identity, cluster metadata—is learned from the first contacted peer via the Raft protocol itself.

3.5.2 SRV record format

An SRV record has the form:

```
1 _service._proto.name. TTL IN SRV priority weight port target.
```

Listing 12: SRV record format

For a three-node Raft cluster in the `raft.local` zone:

```
1 _raft._tcp.raft.local. IN SRV 10 10 7000 raft-1.raft.local.  
2 _raft._tcp.raft.local. IN SRV 10 10 7000 raft-2.raft.local.  
3 _raft._tcp.raft.local. IN SRV 10 10 7000 raft-3.raft.local.
```

Listing 13: Example SRV zone entries

All three entries have equal priority (10) and weight (10) since all Raft voters are equally valid bootstrap contacts. The port (7000) is the Raft TCP port each node listens on.

3.5.3 Startup sequence

When a node starts with `RAFT_CLUSTER_SRV` set, it executes the following sequence:

1. Bind the local Raft port.
2. Resolve `RAFT_CLUSTER_SRV` via DNS (using `libresolv` in C++, `JNDI` in Java, `system dig` in Rust).
3. Shuffle the returned SRV targets to avoid all nodes contacting the same seed.
4. Connect to each target in turn until one responds with a valid Raft handshake.
5. From that contacted peer, learn: the current term, the current leader, and the committed cluster membership.
6. Stop treating DNS as authoritative—subsequent peer discovery is driven by Raft membership log entries, not repeated SRV lookups.

If no seed answers and `RAFT_BOOTSTRAP_NEW_CLUSTER` is not set, the node fails fast. This prevents a transient DNS failure from accidentally forming a split-brain cluster. Bootstrapping a new cluster must be explicit.

3.5.4 Implementation across languages

All three implementations support DNS SRV bootstrapping, though the resolution mechanism differs:

Java `DnsSrvSeedProvider` resolves via JNDI DNS lookups (`javax.naming.directory`). Set `RAFT_CLUSTER_SRV` as a system property or environment variable. The Java adapter reads it from `RuntimeConfiguration`.

C++ `DnsSrvSeedProvider` uses the system resolver (`res_query` from `libresolv`). The C++ CLI reads `RAFT_CLUSTER_SRV` from the environment. The CMake build links `resolv` on Unix.

Rust `DnsSrvSeedProvider` shells out to `dig +short SRV with a fallback to nslookup -type=SRV`. The Docker entrypoint (`entrypoint-rust.sh`) reads `RAFT_CLUSTER_SRV` and calls `dig` to build the peer list before launching `graft-kv`.

In Docker Compose, the Rust entrypoint is used with explicit `RAFT_PEERS` by default, which avoids the `dig` dependency in the container. Adding `dnsutils` to the Rust Dockerfile enables the SRV path. The Java and C++ images support SRV natively without extra packages.

3.5.5 Kubernetes / OpenShift alignment

The DNS SRV model maps directly to Kubernetes headless Services:

```
1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: raft
5 spec:
6   clusterIP: None           # headless
7   selector:
8     app: raft
9   ports:
10    - name: raft
11      protocol: TCP
12      port: 7000
13      targetPort: 7000
```

Listing 14: Kubernetes headless Service

With a `StatefulSet` backing this Service, Kubernetes creates SRV records automatically. Each node sets `RAFT_CLUSTER_SRV=_raft._tcp.raft.<ns>.svc.cluster.local` and the platform handles DNS propagation. This gives a clean portability boundary: locally you own the CoreDNS zone file; in OpenShift, the platform owns the DNS records.

3.6 Where to go next

- Section 5 for the domain application boundary.
- Section 6 for what Raft guarantees—and does not guarantee—under partitions and failovers.
- Section 7 for a comparison with XATMI/XA transaction models, useful if you are coming from a traditional middleware background.
- Section 8, 9, or 10 for language-specific integration details.
- Section 13 for test workflows and mixed-smoke checks.

3. *GETTING STARTED*

- `raft-app-kv/`, `graft-cpp/src/app/`, or `raft-rust/raft-app-kv/` for reference implementations.

4 System Overview

The repository contains one shared protocol and three implementation tracks:

Java Maven modules implementing the main Raft runtime, storage, transport, state-machine contracts, example applications, telemetry and tests.

C++ A CMake subtree under `graft-cpp` implementing the same wire protocol and a bounded but growing Raft runtime using Boost.Asio.

Rust A Cargo workspace under `graft-rust` implementing the same wire protocol and a functionally equivalent Raft runtime using Tokio.

Shared wire schema Protobuf messages under `raft-wire/src/main/proto` used by Java, C++ and Rust.

Jepsen Workload and fault-injection tests under `jepsen`, including mixed Java/C++/Rust scenarios.

4.1 Java module map

Module	Responsibility
<code>raft-wire</code>	Protocol model, protobuf mapping and shared wire messages.
<code>raft-core</code>	Raft node state machine: terms, votes, log matching, commit, snapshot handling, read safety and membership application.
<code>raft-membership</code>	Cluster configuration model and joint-consensus majority calculations.
<code>raft-state-machine</code>	Application state-machine interfaces: <code>SnapshotStateMachine</code> , <code>QueryableStateMachine</code> and result-returning variants.
<code>raft-storage</code>	In-memory and file-backed persistent state and log stores.
<code>raft-transport-netty</code>	Netty transport client/server for Raft and client RPCs.
<code>raft-runtime</code>	Application bootstrap, adapter, operational handlers and policy hooks.
<code>raft-app-kv</code>	Basic key-value demo application.
<code>raft-app-reference</code>	Reference-data application example and policy behavior.
<code>raft-telemetry</code>	Telemetry exporter support.
<code>raft-dist</code>	Runnable distribution entry point.
<code>raft-tests</code>	Unit, integration and cross-module tests.

4.2 C++ layout

The C++ subtree is intentionally smaller but follows the same conceptual layers:

include/graft/core Raft node, application state-machine and snapshot codec.

include/graft/runtime Runtime and RPC handler abstractions.

include/graft/storage Persistent state store.

include/graft/transport Boost.Asio client/server transport.

include/graft/telemetry Prometheus, OTLP and OTEL JSON log export.

src Implementations of the core, runtime, storage, transport, telemetry and CLI.

tests Catch2 tests for the C++ core and runtime behaviors.

4.3 Rust layout

The Rust workspace is structured as seven crates mirroring the Java module boundaries:

graft-proto Generated protobuf types and varint32 envelope codec.

graft-core Transport-agnostic Raft consensus engine.

graft-storage In-memory and file-based log and persistent state stores.

graft-transport Tokio-based TCP client/server with envelope framing.

graft-runtime Active event loop, RPC handler, membership encoders, seed providers, policy hooks.

graft-telemetry Prometheus scrape endpoint, OTLP push and OTEL JSON log export.

graft-app-kv Demo key-value state machine and the `graft-kv` CLI binary.

graft-tests Integration tests covering elections, replication, snapshots, membership, and persistence.

4.4 Boundary principle

The central design rule is:

Raft owns replication safety. The application owns command meaning.

That has practical consequences:

- Raft persists terms, votes, log entries, commit index, membership and snapshot metadata.
- The application persists its own state only through deterministic command application and application snapshot bytes.
- Cluster configuration is part of Raft state, not part of the domain model, although it is persisted beside the replicated application data path.
- Application code should not decide majorities, refresh read leases, send vote requests or mutate the Raft log directly.

4.5 Shared protocol

Java, C++ and Rust interoperate because they share:

- protobuf message definitions,
- envelope framing,
- client command and query request/response shapes,
- membership and join/reconfiguration messages,
- telemetry and cluster-summary messages,
- snapshot install messages.

Cross-language compatibility should be tested at the protocol boundary. If Java, C++ or Rust disagree about status strings, read-safety conditions, membership majorities or snapshot metadata, that is a library bug unless the difference is explicitly documented as domain-specific.

5 Application Developer Guide

This section is for developers who want to use the library to give an application Raft cluster behavior.

5.1 What an application must provide

An application supplies four things:

1. A command encoding for writes.
2. A deterministic state machine that applies committed commands.
3. A snapshot encoding for the application state.
4. Optional query handling for local reads after Raft has established read safety.

The application does not implement leader election, quorum calculation, log replication or membership transitions.

5.2 Command design

Commands are opaque byte payloads to Raft. They should be designed as a stable application protocol:

- Use explicit command types rather than positional assumptions.
- Include stable command identifiers when client retries must be idempotent.
- Keep commands deterministic: all replicas must produce the same state from the same ordered log.
- Avoid embedding local time, random values or local host state unless those values are part of the command chosen before replication.
- Treat command formats as versioned public contracts once multiple deployed nodes may run different software versions.

For reference-data style applications, typical commands are `UPSERT_PRODUCT`, `REMOVE_PRODUCT`, `UPSERT_VARIANT` and `REMOVE_VARIANT`. For a key-value application, commands may be `PUT`, `CAS`, `DELETE` and `CLEAR`.

5.3 State-machine contract

The state machine is called only for committed entries. It should therefore be written as pure replicated state mutation, not as a client-facing validation service. Validation that decides whether a write is accepted before entering Raft belongs in admission, authentication or authorization policy. Validation that is part of deterministic domain semantics may happen inside the state machine.

```
1 public interface SnapshotStateMachine {
2     void apply(long term, byte[] command);
3     byte[] snapshot();
4     void restore(byte[] snapshotData);
5 }
6
7 public interface QueryableStateMachine extends SnapshotStateMachine {
8     byte[] query(byte[] request);
```

```
9 }
```

Listing 15: Java state-machine shape

The C++ interface carries the same idea using C++ types:

```
1 class ApplicationStateMachine {
2 public:
3     virtual std::string apply(std::int64_t index,
4                               std::int64_t term,
5                               std::string_view command) = 0;
6     virtual std::string snapshot() const = 0;
7     virtual std::string query(std::string_view request) const = 0;
8     virtual void restore(std::string_view snapshot) = 0;
9 };
```

Listing 16: C++ state-machine shape

5.4 Snapshots

Application snapshots contain application state only. Raft wraps that payload with Raft metadata, including the cluster configuration at the snapshot boundary. This distinction matters because a snapshot restore must restore both:

- the application’s lookup state,
- the Raft membership and log boundary that make the snapshot safe.

Application developers should not store cluster configuration inside their domain snapshot unless the domain itself has a reason to display or audit it. Cluster configuration is Raft machinery state.

5.5 Reads and read leases

Applications may expose read queries through a queryable state machine. The application should treat such reads as local state inspection. It should not try to contact peers or decide whether the local node is a safe reader.

The runtime decides whether a read may be served:

- Followers redirect ordinary linearizable reads to the known leader.
- A leader may serve a read if its quorum-backed read lease is fresh.
- If the lease is stale, the leader must refresh a quorum barrier before serving.
- If the barrier cannot be refreshed, the response should ask the client to retry.

5.6 Client-facing policies

The runtime has policy hooks around client commands:

Admission Should this local node admit the write at all?

Authentication Who is making the request?

Authorization Is the authenticated principal allowed to perform this write?

These hooks are application-facing. They are not Raft safety hooks. A policy may reject a write for domain or security reasons, but it must not claim a command is committed. Only Raft commit and state-machine application can do that.

5.7 Cluster configuration storage

Applications do not need a separate domain table or object for cluster configuration. Configuration is replicated through Raft internal log entries and is captured in Raft snapshot metadata. The library persists it through the same Raft persistence path as terms, votes and log state.

The important application responsibility is operational: the deployment must provide durable storage for the Raft node. If the node loses its Raft storage it may lose its term, vote, log and membership history. That is a cluster operational failure, not a domain state-machine feature.

6 Consistency Boundaries

This section explains what the Raft implementation guarantees when the network partitions and the cluster appears to split into disconnected groups. It separates two concerns:

1. consistency of the replicated Raft log and the state machine driven by that log—Raft addresses this directly,
2. consistency of everything around that log—clients, retries, caches, projections, and external side effects—which remains an application and integration responsibility.

6.1 What “split brain” means here

In informal distributed-systems language, “split brain” usually means a network partition where the cluster is divided into separate sets of nodes where more than one set of nodes behaves as if it is the authoritative cluster. In a “split brain” scenario, what we fear is that writes could proceed independently in individual sets, each believing they are *the* cluster, and then even after healing the data sets would diverge.

A correct Raft system, guarantees that this outcome can not happen for committed state.

What *can* happen under a partition:

- The cluster partitions into a majority side and one or more minority sides.
- Nodes on multiple sides may temporarily believe they should lead.
- An old leader may continue acting as leader until it notices quorum loss.
- Clients may see redirects, retries, timeouts, or ambiguous outcomes.

What should *not* happen:

- Two different partitions both committing conflicting log entries for the same log position in the same configuration.
- A minority partition continuing to commit authoritative writes as if it still had quorum.

The useful distinction is: apparent split brain at the transport or liveness level can happen; committed split brain at the replicated-log level should not.

6.2 Why “majority” and “minority” are well-defined

The terms majority and minority are not decided dynamically by “who can currently see whom.” In Raft, elections and commitment are evaluated against the configured cluster membership—a specific current voting set. A candidate wins only by receiving votes from a majority of that configured set; an entry commits only when replicated to a majority of that configured set.

If a 5-node voting configuration {A, B, C, D, E} has quorum threshold 3, and the network partitions into {A, B, C} and {D, E}, then {A, B, C} is the majority side and {D, E} is the minority side. This holds regardless of whether D and E can still reach each other. Reachability affects liveness; configuration defines legitimacy.

The same logic applies to any partition shape. A 7-node cluster split into 4 + 3: only the 4 side can make committed progress because quorum is 4. The cluster cannot legitimately shrink itself by accident just

because nodes disappeared behind a partition. A new voting set only becomes authoritative after the cluster commits that configuration change through the log.

6.3 What Raft guarantees

For the authoritative replicated state machine, Raft gives these safety properties:

- Only a leader with a quorum can commit new log entries.
- Once an entry is committed, future leaders must contain that entry.
- Conflicting uncommitted entries on losing sides are eventually overwritten.
- After partition healing, followers converge to the majority-selected log.
- Membership changes are serialized through the log and protected by joint consensus.

Applied to this implementation:

- Ordinary writes are accepted only through the leader path.
- Write success is defined as committed and applied, not merely appended locally.
- Linearizable reads are served only by a leader that can establish a fresh quorum-backed read window.
- Removed nodes decommission themselves after finalized removal.

In practice, under a partition: the majority side may continue to elect a leader and commit writes; the minority side may be alive but should lose the ability to commit new authoritative writes; once the partition heals, minority-side uncommitted divergence is discarded.

The Jepsen partition scenarios (Section 14) validate this: isolating one node, isolating the leader, cutting the leader into a minority, and overlapping reconfiguration with a partition—all checking that only one side keeps making committed progress.

6.4 What Raft does not guarantee

Raft does not automatically make the rest of the application ecosystem consistent. It does not solve:

- client retries after ambiguous timeouts,
- duplicate command submission,
- side effects that happen before commit is known,
- cache invalidation across failover,
- stale read models or projections,
- reconciliation of data written outside the Raft leader path,
- domain-level merge policy for concurrent offline edits,
- idempotency of calls into external systems.

This is the point often mislabeled as “eventual consistency.” Inside the Raft state machine, the goal is strong consistency for committed state. Around the state machine, many components are only eventually consistent: materialized views, denormalized read stores, caches, search indexes, downstream analytics pipelines, webhook consumers, external billing or provisioning systems. Those must converge eventually, but Raft alone does not define how.

6.5 The three outcome classes

When a client issues a command during instability, there are three fundamentally different outcomes:

Definitely committed The leader had quorum, the log entry committed, and the state machine applied it. The clean case.

Definitely not committed The request was rejected because the node was not leader, the cluster had no usable leader, the request was invalid or unauthorized, or the CAS precondition did not match. Safe to treat as no state change.

Ambiguous The client timed out, disconnected, or lost contact while the cluster was unstable. The command may have committed, or it may have been dropped. The client cannot safely assume either outcome.

The ambiguity exists even in a correct Raft system because the client can lose the response after the cluster has already committed the command. The Jepsen harness treats such outcomes as indeterminate rather than definite failures for this reason.

6.6 Where inconsistency can still appear

6.6.1 Uncommitted local divergence

A minority or superseded leader may append entries locally that never commit. That local log tail is inconsistent with the surviving majority for a while. Raft handles this by rollback and overwrite during reconciliation. Application state changes driven only by committed entries do not need custom business logic for this case.

6.6.2 Side effects emitted too early

If application code sends an email, charges a card, provisions a resource, or updates another datastore before the Raft command is known to be committed, a partition or failover may produce an external side effect with no corresponding committed entry in the authoritative log. Raft cannot repair that automatically.

6.6.3 Duplicate execution from retries

If a client times out and retries without an idempotency key, the cluster may eventually apply the same logical intent twice. Raft preserves log consistency, but not business-level uniqueness unless the application models it.

6.6.4 Stale or lagging read models

If a system derives secondary views from the Raft log asynchronously, those views may lag after partitions, failovers, or restart recovery. That is not a Raft safety failure; it is an eventually consistent projection.

6.6.5 Writes outside the leader path

If any service allows local writes on followers, offline replicas, edge nodes, or separate databases not serialized through the Raft leader, then conflict resolution becomes an application concern. At that point you are

no longer relying on Raft alone for authoritative writes.

6.7 What application developers must accommodate

6.7.1 Treat authoritative state as log-derived

The cleanest model: only committed Raft log entries mutate authoritative state. Everything else is derived from or synchronized to that state. This prevents most split-brain reconciliation problems.

6.7.2 Use idempotency keys for client commands

Clients should attach a stable command identity for operations that may be retried. This lets the application distinguish “same logical command retried after timeout” from “new logical command.” Without that, ambiguous client failures can become duplicate business actions.

6.7.3 Delay irreversible side effects until commit is known

If a side effect is not reversible, it should not happen before the corresponding Raft command is committed and applied. Safer patterns: commit first, then emit side effects from an outbox or committed-event stream; or emit only compensatable side effects.

6.7.4 Make side-effect consumers replay-safe

Anything downstream of the committed log should tolerate duplicate delivery, delayed delivery, and re-ordered observation across independent streams. This is standard outbox and projection discipline.

6.7.5 Define reconciliation rules for external projections

For caches, search indexes, or read models, developers need to define how staleness is detected, how a projection catches up after outage, whether rebuild-from-log is supported, and whether consumers should prefer monotonic versions or log indexes.

6.7.6 Define domain merge rules if offline mutation is allowed

If the product intentionally allows updates outside the single Raft leader path, then Raft no longer provides the whole answer. Developers must define last-writer-wins or version-based conflict rules, per-field merge behavior, tombstone/delete precedence, and human review workflows for irreconcilable conflicts.

6.8 Recommended documentation for applications

Any real application using this Raft layer should document:

1. which data is authoritative and log-backed,

2. which APIs return definite success, definite failure, or ambiguous outcome,
3. how client retries are deduplicated,
4. when side effects are emitted relative to commit,
5. how projections rebuild or catch up after outage,
6. whether any writes occur outside the Raft leader path,
7. what conflict resolution policy applies if they do.

If those items are left implicit, teams often assume Raft solves more than it actually does.

6.9 Bottom line

Raft prevents committed split brain for the authoritative replicated log. Raft does not eliminate ambiguous client outcomes during failures, duplicate intent from retries, stale projections, premature external side effects, or domain conflicts created outside the leader/commit path.

The right mental model: Raft gives strong consistency for committed state; everything around that state still needs explicit application-level consistency design.

7 Raft and XATMI Transaction Thinking

This section is for application developers and architects familiar with XATMI, XA, two-phase commit, service transactions, or similar middleware transaction models. Raft and XATMI can look similar from a distance because both coordinate distributed work, but they are not the same mechanism.

7.1 Short version

Raft answers: *Which command is committed at log index N , in what order, according to a quorum?*

XATMI/XA answers: *Can this transaction commit atomically across a set of enlisted services?*

Raft is a replicated state-machine consensus mechanism. XATMI is an application service and transaction coordination model. In this project, a Raft application command is closer to a durable, replicated service request than to a full XA transaction.

7.2 Where they are similar

Both models have a request that crosses process boundaries (`client` $\rightarrow$ `service/runtime` $\rightarrow$ `coordinated outcome` $\rightarrow$ `response`) and both must handle timeouts, retries, process crashes, network partitions, durable recovery, duplicate delivery, unclear client outcome, and authentication/authorization before mutation.

Both force the application to answer the same practical question: if the client did not receive the response, did the operation happen?

7.3 Where they differ

7.3.1 Raft commits log entries

Raft takes an application command and appends it to a replicated log. Once the leader has replicated the entry to a quorum, the entry is committed. Every correct replica applies committed entries in the same order. The replicated log is the serialization point.

7.3.2 XATMI/XA coordinates resources

XATMI-style transaction processing coordinates a unit of work across services and resource managers. XA/two-phase commit asks all enlisted resources to prepare and then commit or roll back. The transaction coordinator tries to make multiple independent participants agree on one atomic outcome.

7.3.3 The core difference

Raft: one replicated log orders commands for one replicated state machine.

XATMI/XA: one transaction coordinates multiple participants/resources.

Raft gives you linearizable replication of a state machine. It does not automatically give you an ACID transaction across arbitrary external systems.

7.4 Commit semantics

7.4.1 Raft commit

A Raft command is committed when durably replicated to a quorum. After commit, replicas apply it in log order. The application does not decide whether an entry is committed—Raft does. `apply()` is called only for committed log entries.

7.4.2 XA commit

An XA transaction commits when all resources have successfully passed the transaction protocol. If a resource cannot prepare or commit, the transaction manager must resolve the outcome using XA recovery rules. A Raft application state machine does not participate in prepare/commit/rollback.

7.5 Ambiguous outcomes

Both models have ambiguous client outcomes. A client sends a request, the leader commits the command, the leader crashes before the response reaches the client, and the client sees a timeout. From the client's perspective, the operation is uncertain.

The remedy is the same design discipline in both models: assign a stable request id, store enough outcome information to recognize retries, and make retry behavior idempotent from the client's perspective.

7.6 Idempotency and request identity

Application developers should treat request identity as part of the domain command model. A good command shape includes a unique `commandId`, the `requesterId`, the operation type, the domain identifier, and the payload. The state machine keeps a small outcome table mapping `commandId` $\rightarrow$ `result`.

On first application: if the `commandId` is unknown, apply the domain mutation, store the result, and return it. On retry: if the `commandId` is known, return the previously stored result. This pattern is useful in both Raft and XATMI-style systems and is especially important when clients retry after timeout.

7.7 Determinism

Raft application commands must be deterministic. Every node applies the same committed command at the same log index; every node must produce the same state transition. Avoid inside `apply()`: local wall-clock decisions, random number generation, reading local files not replicated by Raft, calling external services, checking local node identity, or using non-deterministic iteration order.

Prefer: include timestamps in the command if the business event needs one, include generated ids in the command, validate before submission, and make `apply()` a pure transition over current replicated state plus command bytes. XATMI services can also suffer from non-determinism, but Raft makes it stricter because all replicas must replay the same command stream.

7.8 External resources

The safest Raft application state machine does not call external systems during `apply()`. If `apply()` calls a database, message broker, REST service, or mainframe transaction service, the operation is no longer just a replicated state-machine transition—you have introduced another participant with its own failure and recovery semantics.

7.8.1 Recommended pattern: Raft as source of truth

For reference data and masterdata: a management system validates a change, submits a command to Raft, Raft commits it, the state machine updates local replicated state, and local readers use fast in-memory lookups. The authoritative distributed value is the Raft log and state machine. External stores can feed it or observe it, but should not be part of the committed application transition unless deliberately modeled.

7.8.2 Outbox pattern

If a committed Raft command must trigger external side effects, use an outbox approach: the `apply` updates replicated domain state and records a pending side effect in replicated state; a separate worker observes the pending side effect, performs the external action idempotently, and submits a follow-up command marking the side effect complete. This keeps Raft `apply` deterministic and gives side effects a recoverable state.

7.8.3 Avoid hidden XA inside Raft `apply`

Do not casually start an XA transaction inside `apply()`, update an external database, call a remote service, and commit the XA transaction. That design combines Raft commit with a second transaction protocol. It may be correct in a specific architecture, but it is no longer a simple Raft state machine and needs a full failure analysis.

7.9 Mapping concepts

XATMI/XA concept	Raft-related concept	Important difference
Service request	Client command/query	Raft commands are replicated before application mutation.
Transaction commit	Log entry commit	Raft commit is quorum replication, not multi-resource prepare/commit.
Resource manager	Application state machine	State machine should be deterministic and local.
Transaction id	Command/request id	Application should model idempotency explicitly.
Timeout	Timeout	Both can leave client outcome ambiguous.

Recovery log	Raft log and snapshots	Raft log orders the replicated state machine.
Rollback	Compensating commands	Raft commands are not rolled back after commitment; compensating commands are used.
Service routing	Leader redirect / client retry	Writes must reach the leader.
Read-only service	Linearizable query	Raft read safety must be established first.

7.10 Rollback and compensation

In XA, rollback is part of the transaction protocol before commit. In Raft, once an entry is committed and applied, the application does not roll it back. If the business process needs reversal, submit a new compensating command. That compensation is itself a new committed fact in the log.

7.11 Reads and lookup workloads

XATMI applications often think in terms of service calls. In this Raft design, the target use case is often few writes, many reads. For reference data and masterdata distribution: central governance produces low-rate replicated changes through Raft; high-rate local lookups are served from domain state. A local lookup is safe only after Raft has established that the node can serve the read. The runtime does that before calling the application's query method. Application developers should not implement quorum checks in the query code.

7.12 Cluster configuration is not domain data

Cluster configuration includes current voters, next voters during joint consensus, learners, membership transition state, and quorum rules. That state belongs to Raft. It is not reference data, not masterdata, not a key/value entry owned by the application. The application snapshot stores business state; the Raft snapshot wrapper stores cluster configuration. Applications must not reserve keys such as `_raft/config` or `_cluster/voters`—those would blur the safety boundary.

7.13 Transaction boundaries for domain developers

When designing a domain command, ask:

1. What is the smallest deterministic state change?
2. What command id identifies this request across retries?
3. What result should be returned if the same command id is seen again?
4. What state must be included in snapshots?
5. What external side effects are needed, if any?
6. Can side effects be modeled as follow-up commands?
7. What authorization/admission rule applies before the command enters Raft?

Do not ask the application to decide: who the leader is, what quorum means, whether a membership change is committed, where cluster configuration is stored, or whether a read lease is valid. Those are Raft runtime questions.

7.14 Design guidance

Use Raft when: the application can be represented as a deterministic state machine, the replicated state is small or compactable enough for snapshots, writes are lower volume than reads, local read performance matters, and the system benefits from clear leader/quorum semantics.

Use XA/XATMI-style coordination when: one business transaction must atomically update multiple independent resource managers, participants cannot be represented as one replicated state machine, and prepare/commit/rollback across external systems is the actual requirement.

Use both only with explicit design: Raft for replicated state, XA/XATMI for external systems that truly require transaction coordination, idempotent bridges between them, and clear recovery and compensation rules.

7.15 Practical rule

For a domain application developer using this project: your command is the transaction-like business request; Raft makes that request durable, ordered, replicated, and recoverable; your state machine makes the deterministic business change; Raft cluster configuration stays outside your domain data.

8 Java Library

The Java implementation is a Maven reactor. Application developers normally interact with the runtime and state-machine modules rather than constructing `RaftNode` directly.

8.1 Application integration path

A Java application should start from:

- `SnapshotStateMachine` or `QueryableStateMachine` in `raft-state-machine`,
- `BasicAdapter` and application module hooks in `raft-runtime`,
- a transport factory, normally Netty through `raft-transport-netty`,
- storage from `raft-storage`.

The normal shape is:

1. Define application command and query payloads.
2. Implement the state machine.
3. Create an adapter or module that supplies that state machine to the runtime.
4. Configure peers, local identity, storage and transport.
5. Let the runtime expose client command/query and operational endpoints.

8.2 State machine interfaces

`SnapshotStateMachine` is the minimal contract. It receives committed commands and can produce/restore application snapshots. `QueryableStateMachine` adds local query support. The result-returning state-machine variant is useful when a committed command should return a typed application result, for example CAS.

State-machine implementations should be thread-conscious. Raft calls are synchronized at the node level where necessary, but application state should still avoid exposing mutable internal collections directly.

8.3 Runtime modules

`RaftApplicationModule` separates application packaging from the shared runtime. It lets an application declare supported runtime modes, CLI commands and an adapter factory. This is where reference-data or other domain packages plug into the generic executable.

`RaftApplicationFactory` creates a runnable adapter for a local peer. It receives:

- local peer identity,
- initially known peers,
- optional join seed,
- runtime configuration,
- timeout settings.

An application should prefer these runtime hooks over direct construction of transport and node internals.

8.4 Policy hooks

The Java runtime exposes:

- `ClientWriteAdmissionPolicy`,
- `ClientCommandAuthenticator`,
- `ClientCommandAuthorizer`.

Use these for domain and security decisions before a command enters Raft. Keep the policy decisions explicit and testable. Do not hide domain rejection inside a state-machine no-op unless that no-op is itself the deterministic domain behavior.

8.5 Storage

The Java storage layer separates persistent term/vote state from the log store. Production deployments should use durable stores. Tests may use the in-memory stores, but those stores are not a recovery mechanism.

Cluster configuration is applied from internal replicated entries and is also carried in snapshot metadata. Application code should treat configuration as Raft-owned state.

8.6 Javadocs and Maven publishing

The reactor is configured to attach source and Javadoc jars. The public API should therefore keep class and method Javadocs useful for application developers. In particular, boundary interfaces should document whether a method is application-owned or Raft-owned.

The usual publishing readiness checks are:

- `mvn-qtest`
- `mvn-q-DskipTestpackage`
- verify source and Javadoc jars under module `target` directories
- ensure publishing metadata and signing are configured before Maven Central upload

9 C++ Library

The C++ implementation lives under `graft-cpp`. It is built separately with CMake so C++ deployments do not depend on the Maven reactor. It shares the protobuf schema and envelope protocol with Java.

9.1 Build model

The C++ build creates:

- `graft_proto`: generated protobuf types,
- `graft_transport`: reusable library target for core/runtime/storage/transport,
- `graft_smoke`: command-line probe and runnable modes,
- `graft_unit_tests`: Catch2 tests when Catch2 v3 is available.

Basic workflow:

```
1 cmake -S graft-cpp -B graft-cpp/build
2 cmake --build graft-cpp/build
3 ctest --test-dir graft-cpp/build --output-on-failure
```

Listing 17: C++ build workflow

The code requires C++20. Older Boost.Asio installations may still reference `std::result_of`; the CMake target defines `BOOST_ASIO_DISABLE_STD_RESULT_OF` to remain compatible with such headers in C++20 mode.

9.2 Application boundary

The C++ application boundary mirrors the Java state-machine boundary:

- commands are opaque bytes or strings at the Raft layer,
- application state machines apply committed commands,
- snapshots are application bytes wrapped by Raft metadata,
- queries are served only after the runtime establishes read safety.

The current C++ example state machine is key-value oriented. Reference-data mode behavior exists at the policy boundary, but the Java implementation remains the fuller reference-data domain implementation.

9.3 Runtime and transport

Boost.Asio provides client and server transport over the shared envelope protocol. The active server modes wire a local C++ `RaftNode` to an RPC handler and transport server. The smoke CLI can also act as a client for Java or C++ peers.

Use C++ smoke commands to validate boundary behavior:

```
1 graft-cpp/build/graft_smoke client-put 127.0.0.1 11082 k v1 cpp-node
2 graft-cpp/build/graft_smoke client-get 127.0.0.1 11082 k cpp-node
3 graft-cpp/build/graft_smoke cluster-summary 127.0.0.1 11082 cpp-cli
```

```
4 graft-cpp/build/graft_smoke reconfiguration-status 127.0.0.1 11082 cpp-cli
```

Listing 18: Selected C++ probe commands

9.4 Telemetry and observability

The C++ implementation supports two layers of telemetry.

9.4.1 In-cluster telemetry (protobuf RPC)

All three telmetry RPC types are handled through the shared wire protocol: `TelemetryRequest/TelemetryResponse`, `ClusterSummaryRequest/ClusterSummaryResponse`, and `ReconfigurationStatusRequest/ReconfigurationStatusResponse`. The `InMemoryRpcHandler` and `PersistentRpcHandler` classes in `src/runtime/rpc_handler.cpp` implement all three with authentication, rate limiting, leader redirects and cluster health summaries matching the Java reference behavior.

9.4.2 External telemetry endpoints

Configured via the `RAFT_TELEMETRY_EXPORTER` environment variable (matching the Java `raft.telemetry.exporter` property):

- `prometheus`: starts a separate HTTP listener on `RAFT_TELEMETRY_PROMETHEUS_HOST:RAFT_TELEMETRY_PROMETHEUS_PORT` (default `127.0.0.1:9108`) and serves Raft metrics in Prometheus exposition format at `RAFT_TELEMETRY_PROMETHEUS_PATH` (default `/metrics`). The listener is implemented directly on top of Boost.Asio without an external HTTP library.
- `otlp/opentelemetry`: configuration is recognized and parsed but push export is not yet implemented (the timer skeleton is in place).
- `otel-log/opentelemetry-log`: periodically writes a single-line JSON snapshot to `std::cout` at the configured `RAFT_TELEMETRY_EXPORT_INTERVAL_SECONDS` interval (default 15 s). The JSON structure matches Java's `OpenTelemetryJsonExporter`.

The telemetry infrastructure lives in `include/graft/telemetry/telemetry_exporter.hpp` and `src/telemetry/telemetry_exporter.cpp`. The exporter is wired into the active server modes in `src/app/cli.cpp`.

9.5 Protobuf toolchain

The C++ build uses `protoc` and `libprotobuf`. They should come from the same package manager prefix. A warning such as compiler version `33.x` versus library version `6.33.x` is usually a CMake version-comparison artifact, but mixed prefixes can cause real compile or link problems. Pin the compiler explicitly when needed:

```
1 cmake -S graft-cpp -B graft-cpp/build \  
2   -DProtobuf_PROTOC_EXECUTABLE=/opt/local/bin/protoc
```

Listing 19: Pinning protoc

9.6 Parity expectations

C++ should be judged against Java at the Raft behavior boundary:

- status values such as `ACCEPTED`, `REDIRECT`, `RETRY` and `INVALID`,
- vote and append semantics,
- snapshot install and restore semantics,
- read barrier and read-lease behavior,
- joint-consensus majority rules,
- membership admission and finalization,
- mixed Java/C++ cluster behavior.

Differences in domain examples can be tolerated temporarily. Differences in Raft safety semantics should be treated as defects.

10 Rust Library

The Rust implementation lives under `graft-rust`. It is a Cargo workspace built with `cargo` so Rust deployments do not depend on the Maven reactor or CMake. It shares the protobuf schema and envelope protocol with Java and C++.

10.1 Build model

The Rust workspace contains eight crates:

- `graft-proto`: shared protobuf generated types plus varint32 envelope framing (byte-compatible with the Java `ProtobufLiteEncoder/ProtobufLiteDecoder` and the C++ `envelope_codec.hpp`),
- `graft-core`: transport-agnostic Raft consensus engine (elections, log replication, commit/apply, snapshots, joint-consensus membership, read barriers),
- `graft-storage`: in-memory and file-based log and persistent state stores,
- `graft-transport`: async TCP client and server with varint-length-prefixed envelope framing,
- `graft-runtime`: active event loop (`RaftRuntime`), message handler, membership encoders, seed providers, policy hooks,
- `graft-telemetry`: Prometheus scrape endpoint, OTLP HTTP push (behind `otlp` Cargo feature), and OTEL JSON log export,
- `graft-app-kv`: demo key-value state machine and the `graft-kv` CLI binary,
- `graft-tests`: integration tests covering elections, log replication, snapshots, membership, and file persistence.

Basic workflow:

```
1 cd graft-rust
2 cargo build -p graft-app-kv      # build the CLI binary
3 cargo test -p graft-tests        # run the integration tests
4 cargo check                     # type-check the entire workspace
```

Listing 20: Rust build workflow

The code targets stable Rust (edition 2021). The build fetches crate dependencies automatically from `crates.io`. The shared `raft-wire/src/main/proto/raft.proto` is compiled by the `graft-proto` crate's `build.rs` using `prost-build`.

10.2 Application boundary

The Rust application boundary mirrors the Java state-machine boundary:

- `StateMachine` trait with `apply`, `snapshot`, and `restore`,
- `QueryableStateMachine` extension for read queries,
- `ResultStateMachine` extension for client-visible results,
- `LogStore` and `PersistentStateStore` traits for storage backends.

The demo `KeyValueStateMachine` implements `StateMachine` and `QueryableStateMachine`. Commands and queries are JSON-serialized into the Raft log entry data field. Domain application developers should implement these traits and register their state machine with a `RaftNode`.

10.3 Runtime and transport

Tokio provides async client and server transport over the shared envelope protocol. The `RaftRuntime` is the active event loop: it runs randomized election timers (1500–3000 ms), broadcasts heartbeats every 750 ms, and processes responses to update `matchIndex/nextIndex` and advance the commit index.

The `RaftHandler` dispatches all 11 Raft RPC types and enforces configurable authentication, authorization, admission, and rate-limiting policies through the policy traits in `graft-runtime/src/policy.rs`.

The active server modes wire a local Rust `RaftNode` to the RPC handler and transport server. The smoke CLI can also act as a client for Java, C++, or Rust peers.

Use Rust smoke commands to validate boundary behavior:

```
1 graft-rust/target/debug/graft-kv client-put 127.0.0.1 11082 k v1 rust-client
2 graft-rust/target/debug/graft-kv client-get 127.0.0.1 11082 k rust-client
3 graft-rust/target/debug/graft-kv cluster-summary 127.0.0.1 11082
4 graft-rust/target/debug/graft-kv reconfiguration-status 127.0.0.1 11082
```

Listing 21: Selected Rust probe commands

10.4 Cross-language mixed smoke tests

Rust and Java mixed smoke scripts mirror the existing C++ and Java scripts:

```
1 graft-rust/tests/scripts/run-mixed-smoke-rust-leader.sh
2 graft-rust/tests/scripts/run-mixed-smoke-java-leader.sh
3 graft-rust/tests/scripts/run-interop-smoke.sh
4 graft-rust/tests/scripts/run-mixed-suite.sh
```

Listing 22: Rust mixed smoke tests

These reuse the Java probe from `graft-cpp/java-probe` and exercise elections, log replication, telemetry, and client redirects across Rust and Java nodes. Run these after changing protobuf messages, status strings, or any RPC behavior.

10.5 Protobuf toolchain

The Rust code uses the same shared `raft.proto` as Java and C++. The `prost` crate generates Rust types at build time. No manual code generation step is needed—it runs as part of `cargo build`. The generated types use Rust protobuf encoding that is byte-compatible with Java `protobuf-lite` and C++ `libprotobuf`.

10.6 Telemetry and observability

The Rust implementation supports two layers of telemetry.

10.6.1 In-cluster telemetry (protobuf RPC)

All three telemetry RPC types are handled through the shared wire protocol in `graft-runtime/src/handlers.rs`: `TelemetryRequest/TelemetryResponse`, `ClusterSummaryRequest/ClusterSummaryResponse`.

and `ReconfigurationStatusRequest/ReconfigurationStatusResponse`. The handlers enforce authentication, rate limiting, leader redirects and cluster health summaries matching the Java and C++ reference behavior.

10.6.2 External telemetry endpoints

Configured via the `RAFT_TELEMETRY_EXPORTER` environment variable:

- `prometheus`: starts an HTTP listener on the configured port and serves Prometheus exposition text. Implemented directly on top of Tokio without an external HTTP library.
- `otlp/opentelemetry`: requires building with `--features otlp` (adds the `request` crate). Pushes OTLP JSON payloads to the configured collector endpoint on a periodic timer.
- `otel-log/opentelemetry-log`: periodically emits a single-line JSON snapshot via `tracing::info!(t "OTEL", ...)`, matching Java's `OpenTelemetryJsonExporter` pattern.

The export interval is controlled by `RAFT_TELEMETRY_EXPORT_INTERVAL_SECONDS` (default 15 s). The telemetry infrastructure lives in the `graft-telemetry` crate at `raft-telemetry/src/lib.rs`.

10.7 Policy hooks

The Rust runtime includes pluggable policy hooks matching the Java `raft-runtime` interfaces:

- `Authenticator` — validates client credentials (default: `AllowAllAuthenticator`, also includes `SharedSecretAuthenticator`),
- `Authorizer` — authorizes commands against a principal (default: `AllowAllAuthorizer`),
- `AdmissionPolicy` — decides accept/redirect/reject for writes (default: `AcceptAllAdmissionPolicy`, also includes `LeaderRedirectAdmissionPolicy`),
- `RateLimiter` — sliding-window rate limiter for telemetry and cluster-summary requests,
- `StuckDetector` — reports prolonged joint-consensus transitions.

Application modules can replace the default policy components by setting the `Arc` fields on `RaftHandler`.

10.8 Docker support

A multi-stage Dockerfile builds the Rust binary inside a `rust:1-bookworm` container and copies it into a slim `debian:bookworm-slim` runtime image. Compose files for Rust-only and mixed Java+C+++Rust clusters are available:

```
1 docker/run-srv-smoke.sh rust           # 3 Rust nodes
2 docker/run-srv-smoke.sh mixed          # 1 Java + 1 C++ + 1 Rust
3 docker/run-srv-suite.sh                # all 4 modes (java, cpp, rust, mixed)
```

Listing 23: Docker Rust modes

Nodes bootstrap from explicit peer lists (`RAFT_PEERS`) or DNS SRV records (`RAFT_CLUSTER_SRV`) via a shell entrypoint.

10.9 Parity expectations

Rust should be judged against Java and C++ at the Raft behavior boundary:

- status values such as `ACCEPTED`, `REDIRECT`, `RETRY` and `NOT_LEADER`,
- vote and append semantics,
- snapshot install and restore semantics,
- read barrier and read-lease behavior,
- joint-consensus majority rules,
- membership admission and finalization,
- mixed Java/C++/Rust cluster behavior.

Differences in domain examples can be tolerated temporarily. Differences in Raft safety semantics should be treated as defects.

11 Raft Internals

This section is for developers changing the Raft machinery itself.

11.1 Core state

A Raft node maintains:

- current term,
- voted-for candidate,
- local role: follower, candidate or leader,
- replicated log,
- commit index and last-applied index,
- known leader,
- cluster membership configuration,
- leader-side replication progress for followers,
- read-lease and heartbeat timing state.

The safety rules are the usual Raft rules: terms are monotonic, a node votes at most once per term, candidates must have sufficiently up-to-date logs, leaders append entries in order, and committed entries must not be lost across leader changes.

11.2 Elections

Election behavior is driven by timeout and vote responses:

1. A follower whose election deadline expires becomes a candidate.
2. The candidate increments term, votes for itself and sends vote requests.
3. A candidate becomes leader only after satisfying the configured majority rule.
4. Any node that observes a higher term steps down.

Membership matters. In stable configuration, election success requires a majority of current voters. In joint consensus, election success requires a majority of current voters and a majority of next voters.

11.3 AppendEntries and commit

Leaders replicate log entries using AppendEntries. Followers reject requests whose previous index/term does not match their log. Leaders backtrack until the matching prefix is found, then append missing entries.

Commit advancement must respect both term and membership:

- The leader should only commit entries known replicated on a required majority.
- Joint consensus requires both current and next majorities.
- Entries from older terms require the normal Raft care: leadership should be established through an entry in the current term before relying on older entries for commitment.

11.4 Read lease and read barrier

Linearizable reads require proof that the leader is still authoritative. The library uses a read-lease/read-barrier model:

- A leader records successful quorum contact through heartbeat/append responses.
- While that evidence is fresh, it may serve local queries.
- If evidence is stale, it must refresh a quorum heartbeat barrier.
- If the barrier fails, the query returns retry/redirect behavior rather than serving stale state.

The same membership rules apply here: joint consensus requires current and next majorities for the barrier to be safe.

11.5 Joint consensus

Membership changes are replicated through internal log entries. A safe transition uses two phases:

1. Enter joint consensus with current and next configurations active.
2. Finalize into the next stable configuration after the joint entry is committed.

During joint consensus, majority calculations must be split, not flattened. A single large majority across the union is not sufficient. The implementation must evaluate current voters and next voters independently.

11.6 Internal commands

Internal Raft commands are log entries whose meaning is Raft-owned rather than domain-owned. Membership changes are the main example. Internal commands must be decoded and applied before or alongside domain state updates so that effective configuration by log index remains correct.

Application developers should not create internal commands directly. Machinery developers should keep internal command encoding versioned and compatible across Java and C++.

12 Wire, Storage and Snapshots

12.1 Wire protocol

All cross-node and client-facing transport uses a shared protobuf schema. The transport envelope carries:

- correlation id,
- term,
- message type,
- protobuf payload.

Java maps protobuf messages through the `ProtoMapper`. C++ generates protobuf classes from the same schema and uses the same envelope framing. When a new message is added, both languages must be updated and mixed smoke coverage should be considered.

12.2 Status vocabulary

Status strings are part of the cross-language contract. Examples include:

ACCEPTED A client command was accepted after commit/apply.

REDIRECT The contacted node is not the leader and can identify a leader.

RETRY The request may be retried, commonly after read-barrier failure.

INVALID The request payload or operation is invalid.

UNAUTHORIZED Authentication or authorization failed.

Avoid introducing language-specific synonyms. If Java says **ACCEPTED**, C++ should not say **OK** for the same behavior.

12.3 Persistent state

Raft persistence includes:

- current term,
- voted-for candidate,
- known leader where applicable,
- log entries,
- snapshot boundary and metadata,
- cluster configuration.

Application state is persisted indirectly by applying committed entries and by snapshotting application state. If a node restarts, it must reconstruct both Raft metadata and application state consistently.

12.4 Snapshot wrapping

Snapshots have two layers:

Application payload Bytes produced by the state machine.

Raft wrapper Metadata such as last included index, last included term and cluster configuration at that boundary.

This wrapper is essential for restart and catch-up. A follower receiving a snapshot must restore application state and update Raft membership/log boundary state consistently.

12.5 Configuration storage

Cluster configuration is not an external domain object. It is part of the Raft log and snapshot metadata. The application may expose cluster configuration through operational APIs, but it should not duplicate configuration as domain state unless it is doing so for audit or display.

12.6 Protocol evolution

When changing the shared protocol:

1. Update the protobuf schema.
2. Regenerate Java and C++ bindings.
3. Update Java mapping and C++ mapping/handling.
4. Add unit tests in both languages where behavior changes.
5. Add mixed smoke or Jepsen coverage when the change affects cross-language behavior.

Prefer additive schema changes. If a field meaning changes, document the version expectations and test mixed-version behavior before relying on it operationally.

13 Testing and Operations

13.1 Test layers

The project uses several layers of verification:

Java unit tests Core election, append, snapshot, membership, read-lease, adapter and storage behavior.

Java transport tests Netty client/server behavior and response handling.

C++ unit tests C++ core, membership, state-machine, snapshot and RPC behavior.

Rust integration tests Rust core, membership, state-machine, snapshot, replication and file persistence behavior.

Mixed smoke tests Java/C++/Rust interoperability at the shared wire boundary.

Jepsen Fault-injection and workload checks for stronger system-level confidence.

Use the smallest test layer that can prove a change, then add broader tests when the change crosses module, language or safety boundaries.

13.2 Java test workflow

```
1 mvn -q test
2 mvn -q -DskipJepsenTests=true test
3 mvn -q -DskipTests package
```

Listing 24: Java test workflow

Unit tests should avoid real Netty resources unless the test is explicitly about transport behavior. Use no-op or in-memory transport fakes for Raft core tests so older platforms with lower file-descriptor limits do not fail while constructing test nodes.

13.3 C++ test workflow

```
1 cmake -S graft-cpp -B graft-cpp/build
2 cmake --build graft-cpp/build
3 ctest --test-dir graft-cpp/build --output-on-failure
```

Listing 25: C++ test workflow

When CMake finds the wrong protobuf tools, pin `protoc`. If Boost.Asio fails around removed standard-library traits, check the C++ compatibility macro described in Section 9.

13.4 Rust test workflow

```
1 cd graft-rust
2 cargo test -p graft-tests           # integration tests
3 cargo test --workspace              # all tests across all crates
4 cargo build -p graft-app-kv        # build the CLI binary
```

Listing 26: Rust test workflow

The integration tests use in-memory stores and exercise elections, log replication, snapshot install, joint-consensus membership transitions, and file-based persistence round-tripping. No external dependencies are required for tests—the entire suite runs offline.

13.5 Mixed Java/C++/Rust checks

The mixed smoke scripts validate that all implementations agree on the shared protocol:

```
1 graft-cpp/run-mixed-smoke.sh           # C++ leader + Java follower
2 graft-cpp/run-mixed-smoke-java-leader.sh # Java leader + C++ follower
3 graft-cpp/run-mixed-membership-smoke.sh # membership with C++ learner
4 graft-cpp/run-mixed-membership-cpp-leader-smoke.sh # membership with C++ leader
5 graft-cpp/run-mixed-snapshot-smoke.sh    # snapshot catch-up
6 graft-cpp/run-mixed-suite.sh            # all C++ mixed scenarios
7 graft-rust/tests/scripts/run-mixed-suite.sh # Rust and Java smoke suite
```

Listing 27: Mixed implementation checks

Run these after changing:

- protobuf messages,
- status strings,
- client command/query behavior,
- membership behavior,
- read-barrier behavior,
- snapshot install or snapshot metadata.

13.6 Jepsen

Jepsen should be used when a change affects safety under concurrency, failure or network partitions. In particular, run or update Jepsen scenarios for:

- leader election changes,
- commit-index changes,
- read-lease/read-barrier changes,
- membership changes,
- mixed Java/C++/Rust cluster behavior,
- client workload semantics such as CAS.

The Jepsen suite is not a replacement for unit tests. It is the system-level check that the library still behaves correctly when timing and partitions are hostile.

13.7 Operational notes

A Raft deployment needs durable per-node storage. Losing local Raft storage can lose votes, terms, log entries and membership state. For application developers, this means the storage path is part of the cluster deployment contract, even if the domain state itself looks like an in-memory hashtable.

Operators should monitor:

- current leader,
- term,
- commit index and last applied index,

- follower replication lag,
- quorum health,
- current and next quorum health during joint consensus,
- stuck reconfiguration age,
- snapshot install events.

These metrics can be consumed through two channels:

In-cluster RPCs `TelemetryRequest`, `ClusterSummaryRequest`, and `ReconfigurationStatusRequest` provide point-in-time operational snapshots over the shared wire protocol. All three implementations handle these RPCs.

External telemetry endpoints Java, C++ and Rust nodes can expose Prometheus scrape endpoints (`RAFT_TELEMETRY`, OTLP push (Java, Rust behind feature flag), and OTEL JSON log output (all three)). Configuration follows the shared `raft.telemetry.* / RAFT_TELEMETRY_*` naming convention. See the library chapters for each implementation's specific exporter support.

13.8 Cross-language parity

The three implementations (Java, C++, Rust) are intended to be functionally equivalent at the Raft boundary. When changing one, verify the others against this checklist:

- Status strings must be identical across implementations: `ACCEPTED`, `REDIRECT`, `RETRY`, `NOT_LEADER`, `INVALID`, `UNAUTHORIZED`, `RATE_LIMITED`.
- Vote-granting rules (term comparison, log up-to-date check) must be identical.
- `AppendEntries` conflict resolution (backtracking, truncation, match-index advancement) must be identical.
- Snapshot install and restore semantics (chunk assembly, metadata wrapping) must be identical.
- Joint-consensus majority rules must use the same quorum formula in all three.
- Membership admission and finalization (`JOIN` and `JOINT` and `FINALIZE`) must be identical.
- Read barriers must contact a joint-consensus quorum of voters before serving.
- Client redirect behavior (leader-id, leader-host, leader-port in responses) must be identical.
- Differences in domain examples (key-value store vs. reference-data) are acceptable.

13.9 Codebase navigation

For machinery developers who need to find specific Raft behaviors across implementations:

Behavior	Java	C++	Rust
Election timeout / jitter	<code>raft-core/.../RaftNode.java</code>	<code>graft-cpp/src/core/raft_node.cpp</code>	<code>graft-rust/raft-core/src/raft_node.rs</code>
<code>AppendEntries</code> follower	<code>RaftNode.java</code>	<code>raft_node.cpp</code>	<code>raft_node.rs</code>
Commit advancement	<code>RaftNode.java</code>	<code>raft_node.cpp</code>	<code>raft_node.rs</code>
Snapshot install (follower)	<code>RaftNode.java</code>	<code>raft_node.cpp</code>	<code>raft_node.rs</code>

13. TESTING AND OPERATIONS

Joint-consensus majority	raft-membership/.../ClusterConfiguration.java	raft-cpp/src/core/cluster_membership.cpp	raft-rust/raft-core/src/membership.rs
Active runtime loop	raft-runtime/.../BasicAdapter.java	raft-cpp/src/runtime/raft_runtime.cpp	raft-rust/raft-runtime/src/runtime.rs
RPC handler dispatch	raft-transport-netty/.../RaftMessageHandler.java	raft-cpp/src/runtime/rpc_handler.cpp	raft-rust/raft-runtime/src/handlers.rs
Transport framing	raft-transport-netty/.../ProtobufLiteEncoder.java	raft-cpp/include/raft/core/envelope_codec.hpp	raft-rust/raft-protocol/src/lib.rs
Membership encoders	ClusterConfigurationCommonHandler.java	raft-cpp/src/runtime/rpc_handler.cpp (inline)	raft-rust/raft-runtime/src/membership.rs
Telemetry (Prometheus/OTLP/OTEL log)	raft-telemetry-export (Prometheus/OTLP/OTEL log)	raft-cpp/include/raft/telemetry/telemetry_exporter.hpp	raft-rust/raft-telemetry/src/lib.rs
ReconfigurationStatus RPC	raft-runtime/.../raftMessageHandler.java	raft-cpp/src/runtime/rpc_handler.cpp	raft-rust/raft-runtime/src/handlers.rs
Auth / admission hooks	raft-runtime/.../ClientWriteAdmissionPolicy.java	raft-cpp/src/runtime/rpc_handler.cpp (Authenticator)	raft-rust/raft-runtime/src/policy.rs
State machine interface	raft-state-machine/.../SnapshotStateMachine.java	raft-cpp/include/raft/core/application_state_machine.hpp	raft-rust/raft-core/src/state_machine.rs
Example KV application	raft-app-kv/	raft-cpp/src/core/key_value_state_machine.cpp	raft-rust/raft-app-kv/src/lib.rs

Changes that affect Raft behavior should be made in all three files at the same row of this table. Unit tests and mixed-smoke scripts should be run for each affected implementation.

14 Jepsen Testing

Jepsen is the system-level fault-injection and linearizability verification framework. It exercises the cluster under hostile conditions: process crashes, network partitions, clock skew, membership changes, and snapshot boundary exercises. This section describes the Jepsen architecture so machinery developers can add new scenarios and debug failures.

14.1 Architecture

The Jepsen harness lives under `jepsen/`. It is a Clojure project that orchestrates local OS processes or Docker containers, generates randomized workloads, injects faults through a *nemesis*, and checks results against a Knossos linearizability model.

Key source files:

src/raft_jepsen/core.clj CLI parsing, test map construction, generator, checker, and workload definitions.

src/raft_jepsen/db.clj Node lifecycle: start, stop, kill, wipe, port resolution, and implementation-specific command-lines (Java, C++, Rust).

src/raft_jepsen/client.clj Workload executor: translates Jepsen operations (`:write`, `:read`, `:cas`) into CLI commands, parses responses, and classifies outcomes.

src/raft_jepsen/nemesis.clj Fault injection: crash-restart, process-pause, clock-skew, persistence-loss, network partitions, snapshot-boundary exercises, and membership changes.

src/raft_jepsen/observer.clj Observability: captures telemetry and cluster-summary snapshots at interesting points for offline analysis.

14.2 Node implementations

The `-node-impls` flag controls which implementation each Jepsen node uses. For a three-node mixed cluster:

```
1 cd jepsen
2 clojure -M:run mixed-all-three-java-leader \
3   --time-limit 8 --concurrency 4 --node-count 3 \
4   --node-impls java,cpp,rust \
5   --client-impl mixed \
6   --cpp-bin ../graft-cpp/build/graft_smoke \
7   --rust-bin ../graft-rust/target/debug/graft-kv
```

Listing 28: Mixed Jepsen run

Each node’s implementation is determined by its position in the `-node-impls` list. The `-client-impl mixed` flag means each Jepsen worker uses the CLI matching its target node’s implementation.

14.3 Workload model

Jepsen generates operations against a CAS (compare-and-set) register model. The register can hold one value at a time; CAS compare-and-swap means “if the current value is X, set it to Y.” Knossos checks that every read returns a value from a linearizable history.

Three operation types are generated:

- :write** Put a key to a value (sent as `client-put`).
- :read** Read the current value of a key (sent as `client-get`).
- :cas** Conditionally write: supply expected and new values (sent as `client-cas`).

14.4 Nemesis catalog

Each nemesis mode exercises a different failure domain:

Nemesis	Behavior
<code>crash-restart</code>	Kill a node and restart it after a configurable interval.
<code>process-pause</code>	Send SIGSTOP/SIGCONT to freeze and resume a node process.
<code>clock-skew</code>	Inject a clock offset into a node's Raft time source.
<code>persistence-loss-restart</code>	Wipe a node's durable state and restart, checking catch-up.
<code>partition-one</code>	Isolate one node from the network.
<code>partition-leader</code>	Isolate the current leader.
<code>partition-leader-minority</code>	Isolate the leader plus one follower (minority partition).
<code>snapshot-boundary-restart</code>	Write a deterministic prelude, crash a node, advance the log past the snapshot threshold, then restart the node.
<code>snapshot-creation-crash</code>	Cyclic crash of a follower during snapshot creation.
<code>snapshot-transfer-crash</code>	Kill/restart cycle during InstallSnapshot transfer.
<code>membership-join-promote</code>	Admit a new learner, promote to voter, finalize.
<code>membership-demote</code>	Demote a voter to learner.
<code>membership-remove-follower</code>	Remove a follower from the configuration.
<code>membership-remove-leader</code>	Remove the current leader.

14.5 Adding a nemesis

To add a new nemesis:

1. Define a nemesis mode string in `core.clj` under the allowed nemesis set and snapshot defaults.
2. Implement the nemesis logic in `nemesis.clj`:
 - `invoke!` — apply the fault to a node or the network,
 - `teardown!` — restore normal operation,
 - `setup!` — optional pre-fault preparation.
3. Register the mode in `core.clj`'s `normalize-opts` so `snapshot-min-entries` and `snapshot-chunk-bytes` are set if needed.
4. Add test cases to `run-suite.sh` with appropriate port bases.
5. Run the new nemesis in isolation first, then as part of the extended or stress suite.

14.6 Reading failure output

Jepsen produces several artifacts in the work directory:

history.edn The full operation history with timing and outcomes.

results.edn The checker verdict (`:valid? true/false`) and per-worker statistics.

timeline.html A visual timeline of operations and nemesis actions.

store/ Jepsen's own durable store for the checker.

observations/ Observer-captured JSONL files with telemetry and cluster-summary snapshots at failure points.

When a test fails, check the `client-anomaly` and `telemetry-parse-failure` entries in the observer JSONL for the cluster state at the moment of failure. The Jepsen log (`jepsen.log`) contains the full process output.

14.7 Suite structure

The test suites in `run-suite.sh` are organized by confidence level:

smoke Fast baseline tests with no nemesis (port bases 10080–10180).

extended 14 cases covering all nemesis modes (port bases 10080–21420).

mixed 25 cases covering Java and C++, Java and Rust, and three-way Java and C++ and Rust clusters (port bases 21480–24130).

stress 9 cases with 300-second runs, concurrency 20, 5000 operations (port bases 22580–23380).

14.8 Docker/SRV Jepsen

The Jepsen harness also supports a Docker backend (`-backend docker-srv`) that orchestrates clusters through Docker Compose instead of spawning local OS processes. This is useful for testing DNS SRV-based bootstrapping and for reproducing failures in a containerized environment.

Supported nemeses in Docker mode are limited to `crash-restart`, `process-pause`, and `partition-*` (network disconnect/connect via `docker network`). Filesystem-based nemeses (`persistence-loss`, `snapshot-boundary`) and `clock-skew` are not supported in Docker mode because they require direct host filesystem access or process-level clock adjustment.

15 Appendix: Commands and Checklists

15.1 Repository build commands

Command	Purpose
<code>mvn-qtest</code>	Run Java tests in the Maven reactor.
<code>mvn-q-DskipJepsenTests=truetest</code>	Run Java tests while skipping Jepsen integration.
<code>mvn-q-DskipTestspackage</code>	Build packages, source jars and Javadoc jars.
<code>cmake-Sgraft-cpp-Bgraft-cpp/build</code>	Configure the C++ build.
<code>cmake--buildgraft-cpp/build</code>	Build C++ targets.
<code>ctest--test-dirgraft-cpp/build--output-on-failure</code>	Run C++ tests.
<code>cdgraft-rust;cargobuild-pgraft-app-kv</code>	Build the Rust CLI binary.
<code>cdgraft-rust;cargotest-pgraft-tests</code>	Run Rust integration tests.
<code>cdgraft-rust;cargocheck</code>	Type-check the full Rust workspace.
<code>graft-cpp/run-mixed-suite.sh</code>	Run mixed Java/C++ smoke checks.
<code>graft-rust/tests/scripts/run-mixed-suite.sh</code>	Run mixed Java/Rust smoke checks.

15.2 Application developer checklist

1. Define command and query payload formats.
2. Implement deterministic command application.
3. Implement application snapshot and restore.
4. Decide whether command results are needed.
5. Decide whether local queries are needed.
6. Add admission/authentication/authorization policy where appropriate.
7. Configure durable Raft storage.
8. Test leader, follower redirect, retry and invalid-payload behavior.
9. Test restart from log and snapshot.
10. Test membership changes if the application will operate dynamic clusters.

15.3 Raft machinery developer checklist

1. Identify whether the change affects safety, liveness, protocol or packaging only.
2. Add focused Java and/or C++ unit tests for the affected rule.
3. Check Java/C++ status vocabulary and response shapes.
4. Check stable and joint-consensus majority behavior.

5. Check read-barrier behavior if reads are involved.
6. Check snapshot metadata and restore behavior if compaction or install is involved.
7. Run mixed smoke tests for cross-language protocol changes.
8. Run Jepsen when failures, partitions, membership or linearizable reads are involved.

15.4 Common failure interpretations

Leader rejects write The contacted node may be follower, learner, decommissioned or unable to commit.

Query returns retry The leader could not refresh a quorum-backed read barrier.

C++ Boost.Asio result_of error Older Boost.Asio was compiled as C++20 without the compatibility macro.

Protobuf compiler/library warning CMake found `protoc` and `libprotobuf` with version strings that do not compare cleanly, often because package manager prefixes differ.

Mixed smoke status mismatch Java and C++ disagree at the protocol contract. Treat this as a parity defect unless explicitly domain-specific.